0. Administration

0.1 Introduction - Existing situation (problem definition)

Academic institutions face growing challenges in efficiently managing learning spaces, particularly classrooms. Despite heavy investment in infrastructure, many rooms remain underutilized while students struggle to find available study spaces. This results in operational inefficiency, wasted resources, and a negative impact on the learning experience.

0.1 Rationale for application development

The main purpose of Active Rooms is to provide real-time availability of classrooms across campus. Students can quickly locate free rooms, while administrators gain insights into usage patterns over time. The system improves resource allocation, reduces costs, and enhances the student experience by promoting flexible, collaborative learning environments.

0.2 Concepts and Definitions

Active Rooms is a smart, location-based application that provides students and administrators with real-time data on classroom availability. The system integrates IoT sensors, interactive maps, and analytics dashboards to improve operational efficiency and learning experiences.

1. System Description

1.1 The Character of the Application

- Web and mobile-based platform to monitor classroom occupancy in real-time
- Uses IoT sensors to track occupancy status.
- Interactive map to display available and occupied rooms.
- Reports and dashboards for administrators.
- English language support
- Secure authentication and role-based access.
- Client-Server architecture (React frontend, Node.js backend, MySQL database).

1.2 System Users

"Student" user - Searches for available classrooms, views room details, and receives notifications about availability.

"Administrator" user – Manages users, has permissions, can add, delete, and edit sensors, rooms, areas, and zones.

1.3 Usage Scenarios (Use Cases)

1.3.1 Student Use Cases

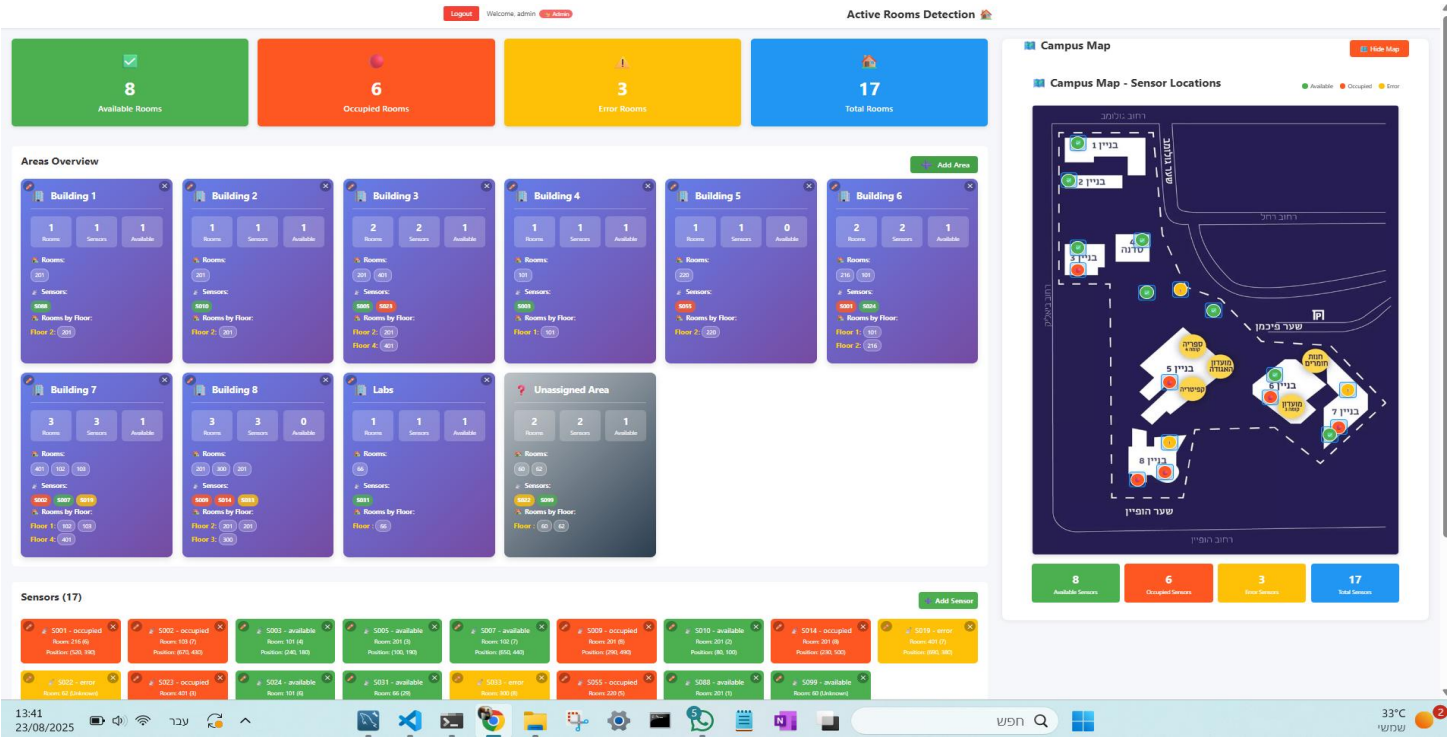
- Login to the application and view the interactive campus map.
- Search for available classrooms based on size, equipment, or location.
- Receive notifications if a previously occupied classroom becomes free.

1.3.2 Administrator Use Cases

- View classroom availability and occupancy statistics.
- Generate usage reports for academic and operational purposes (future).
- Future management of bookings for areas or classrooms in relation to courses or activities.
- Add, update, and remove users with role-based permissions (future).
- Generate system-wide analytics and reports (future).
- Maintain security through encrypted authentication and monitoring.

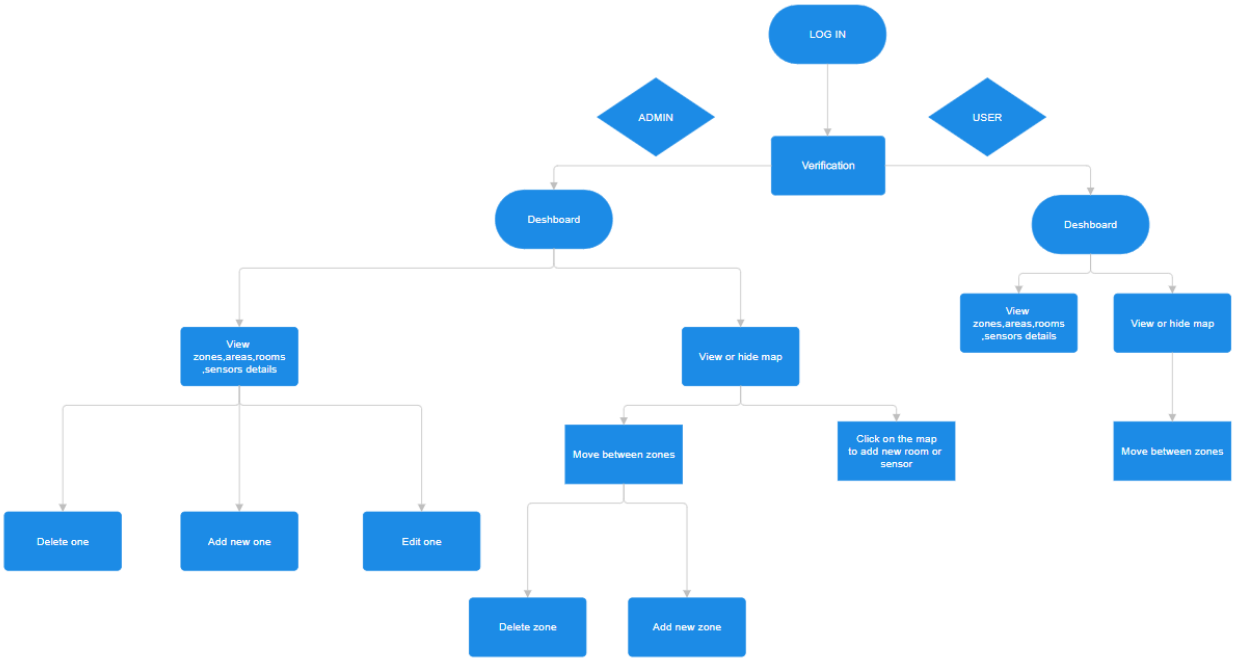
1.3.3 Room Management Menu

- Administrators can add, delete, and edit information.
- Define classroom details such as capacity, equipment, and location (future).
- Sensors automatically update room status (occupied/free).
- Administrators can reserve classrooms for activities.



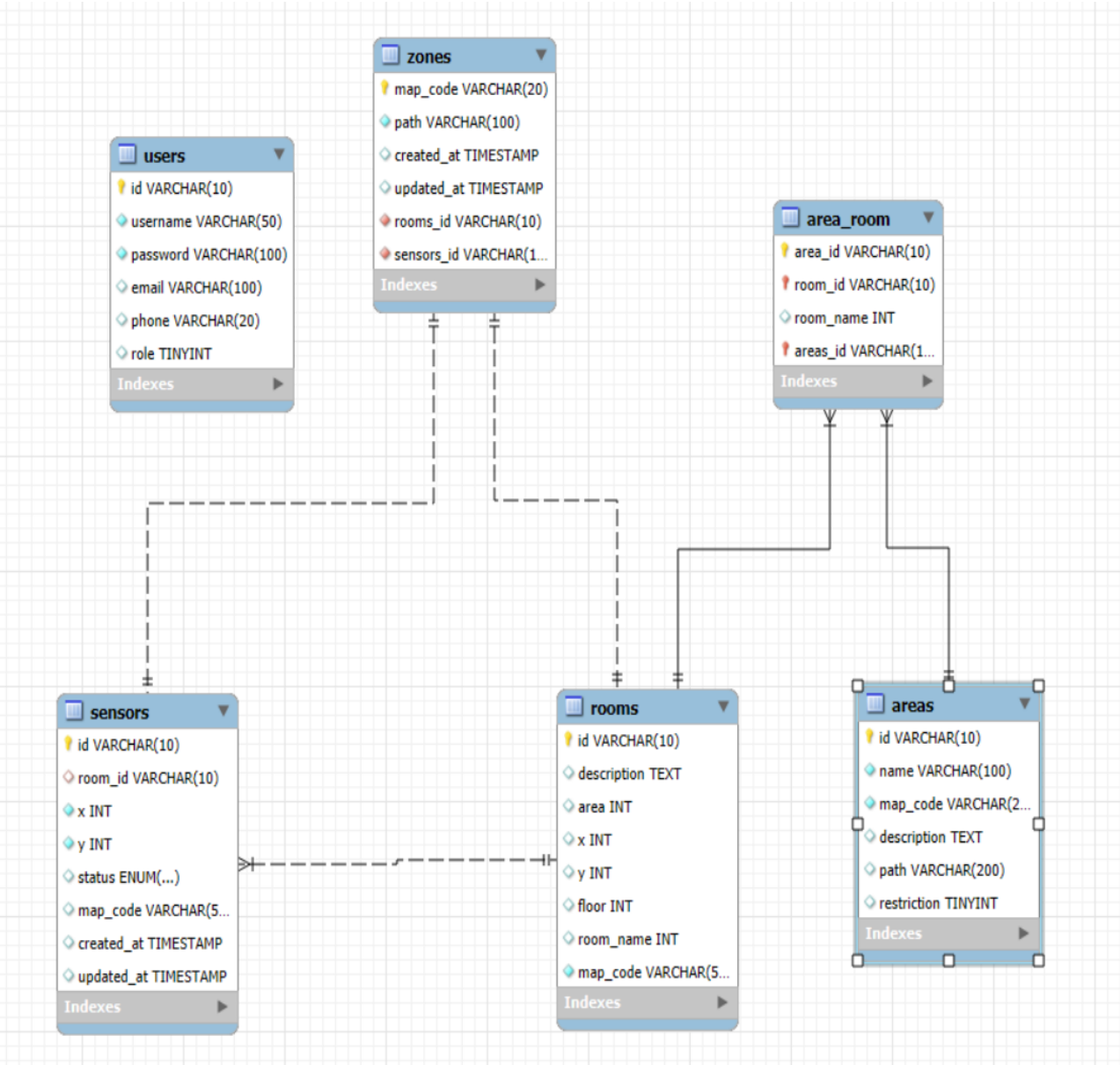
2. Workflow

1. Workflow process:
   Sensors collect occupancy data from classrooms.
2. Data is transmitted to the Backend via a secure API.
3. The Backend processes and updates the MySQL database.
4. The Frontend (React) queries the API and displays the updated results.
5. Students and administrators log in and, according to their permissions, access the relevant data.
6. Students and administrators interact with maps, dashboards, and reports in real-time.
7. When an administrator updates details about a classroom, area, or room in the system, a command is sent to the server to update the database.
8. The server returns a response to the user regarding the success or failure of the action.



3. Database Tables

Users Table: (id, username, password, role)
Rooms Table: (id, name, location, capacity, map_code)
Sensors Table: (id, room_id, status, coordinates, map_code)
Areas Table: (id, name, description, map_code)
Zones Table: (map_code, path, room_id, sensor_id)
Area_Room Table: (id, room_id, area_id, room_name)



4. Non-Functional Requirements

- Performance: Response times under 2 seconds.
- Security: Encrypted passwords (SHA-256 + Salt), HTTPS communication.
- Usability: Responsive, intuitive UI/UX with tooltips, multilingual support.
- Scalability: Support growth up to 5x classrooms and users.
- Compatibility: Works on modern browsers (Chrome, Edge, Firefox) and OS (Windows, macOS, Android, iOS).

5. Architecture & Design

The system uses a client-server architecture:

Frontend: React application with interactive maps, dashboards, and reports.
Backend: Node.js with Express, REST APIs, JWT authentication, validation.
Database: MySQL with indexed tables for optimization.
APIs: REST endpoints for integration and sensor updates.

6. Algorithms

- Real-time sensor data processing: Updates occupancy status every X seconds.
- Coordinate mapping algorithm: Maps sensors to graphical coordinates on the campus map.
- Smart search: Filter classrooms by criteria (size, equipment, location).
- Occupancy rate calculation: (Number of times occupied / Total monitoring time) × 100.
- Security validation: Double validation on client and server side.

7. UI/UX

- Login screen with authentication.
- Dashboard with real-time occupancy statistics.
- Interactive map with color-coded room status.
- Management screens for rooms, areas, and sensors.
- Reports screen for administrators.

8. Testing & Validation

- Unit Testing: Functions such as sensor processing, login, CRUD operations.
- Integration Testing: Communication between Frontend ↔ Backend ↔ Database.
- System Testing: Full workflow validation.
- User Acceptance Testing (UAT): With students, faculty, and administrators.
- Performance Testing: Load tests for 500+ users.
- Security Testing: SQL Injection, XSS, CSRF prevention.

9. Future Development and System Adaptation for Possible Implementations

-Support for additional sensors: Temperature, lighting, CO_2 – enabling monitoring tailored to the needs of different buildings and facilities.

-Advanced analytics and predictive modeling: Using Machine Learning to predict room occupancy and provide recommendations for optimal utilization.

-Mobile app with push notifications: Real-time updates for users depending on the type of institution (students, staff, administrators).

-WebSocket/MQTT support: Real-time updates for sensors and external systems.

-Integration with external systems: BMS, calendars (Google/Outlook) – adapted to existing systems in various buildings.

Possible implementations and system adaptation:

-Educational institutions: Adaptation for monitoring classrooms, lecture halls, libraries, with a user-friendly interface for students and staff.

-Office buildings: Monitoring meeting rooms, shared spaces, and smart allocation based on availability and bookings.

-Shopping centers: Monitoring stores and public areas, tracking occupancy for optimized crowd flow and security.